AWS STRANDS & BEDROCK AGENTCORE

DELTA SUPPLEMENT v2.0

AgentSkills · AgentOps · Strands Labs · Steering · BidiAgent · TypeScript SDK
AgentCore Policy GA · Evaluations · Episodic Memory · Memory Streaming · What's New

Continuation of v1.0	Dec 2025 – Mar 2026	14M+ Downloads	Policy GA · 13 Regions

Version	2.0 — March 28, 2026
Covers	Dec 2025 – March 2026 releases · Strands Labs · AgentSkills · AgentOps · Policy GA
Prereq	AWS Strands & AgentCore Builder Journey Kit v1.0 (companion document)
Audience	AI Architects · Platform Engineers · ML Engineers · Security Teams

DELTA TABLE OF CONTENTS

Supplement to v1.0 Journey Kit

CHAPTER D7 — Updated Best Practices & Anti-Patterns

DELTA CHAPTER D1

Strands Ecosystem Extensions

TypeScript · BidiAgent · Steering · strands_evals · MCP

D1.1 Strands TypeScript SDK — Full Feature Parity

GA · Feb 2026

Strands now ships a first-class **TypeScript SDK** (@strands-agents/sdk) with full feature parity to the Python SDK. This enables type-safe agent development for Node.js, serverless runtimes (AWS Lambda, Vercel Edge), browser environments, and full-stack applications using AWS CDK for infrastructure-as-code.

■ WHAT'S NEW

The TypeScript SDK is ideal for teams building frontend-integrated agents, serverless AgentCore deployments using Lambda, or CDK-native projects. All multi-agent patterns (A2A, Swarm, Graph) and MCP support are available.

agent.ts

```
npm install @strands-agents/sdk
// Basic TypeScript agent — identical mental model to Python
import { Agent, tool } from "@strands-agents/sdk";
const getWeather = tool({
  name: "get_weather",
  description: "Returns current weather for a given city.",
  parameters: { city: { type: "string", description: "City name" } },
  handler: async ({ city }) => `Weather in ${city}: 28°C, sunny`,
});
const agent = new Agent({
  model: "us.anthropic.claude-sonnet-4-20250514",
  systemPrompt: "You are a helpful travel assistant.",
  tools: [getWeather],
});
const response = await agent.invoke("What's the weather in Tokyo?");
console.log(response.message);
// Streaming (SSE-compatible)
for await (const event of agent.stream("Plan a trip to Paris")) {
  if (event.type === "text") process.stdout.write(event.delta);
}
```

D1.2 BidiAgent — Real-Time Bidirectional Audio

Experimental → Stable

The **BidiAgent** enables real-time audio conversations with full duplex streaming — agents listen and respond simultaneously while handling interruptions mid-sentence. Powered by Amazon Nova Sonic v2, this unlocks voice-first agent deployments on AgentCore Runtime with the AG-UI protocol.

bidi_agent.py

```
from strands.experimental.bidi import BidiAgent
from strands.experimental.bidi.models import BidiNovaSonicModel
from strands.experimental.bidi.io import BidiAudioIO, BidiTextIO
from strands.experimental.bidi.tools import stop_conversation
from strands_tools import calculator
import asyncio
# Nova Sonic v2 — production voice model
model = BidiNovaSonicModel(
    provider_config={
        "audio": {
            "input_rate": 16000,
            "output_rate": 16000,
            "voice": "matthew"              # matthew / amy / etc.
        },
        "turn_detection": {
            "endpointingSensitivity": "MEDIUM"  # HIGH/MEDIUM/LOW
        },
        "inference": {
            "max_tokens": 2048,
            "temperature": 0.7
        }
    }
)
agent = BidiAgent(
    model=model,
    system_prompt="You are a helpful voice assistant.",
    tools=[calculator, stop_conversation]
)
async def main():
    audio_io = BidiAudioIO(input_device_index=0, output_device_index=1)
    text_io  = BidiTextIO()
    # Multi-modal: speak OR type, get audio + text back
    await agent.run(
        inputs=[audio_io.input(), text_io.input()],
        outputs=[audio_io.output(), text_io.output()]
    )
asyncio.run(main())
```

bidi_server.py

```
# Server-side BidiAgent on AgentCore Runtime (AG-UI protocol)
# Clients handle audio; server handles agent logic only
pip install strands-agents[bidi]   # No bidi-io needed server-side
from bedrock_agentcore.runtime import serve_ag_ui
from strands.experimental.bidi import BidiAgent
from strands.experimental.bidi.models import BidiNovaSonicModel
bidi_agent = BidiAgent(model=BidiNovaSonicModel(), tools=[...])
serve_ag_ui(bidi_agent)   # Exposes SSE + WebSocket on AgentCore Runtime
```

D1.3 Strands Steering — Middleware for Agent Behaviour

Steering is a modular plugin system that intercepts the agent loop at specific lifecycle hooks — like middleware for HTTP requests, but for agent reasoning. Instead of front-loading every instruction into a massive system prompt (which agents may ignore by line 40), Steering injects context-aware guidance *exactly when needed*, without hardcoding workflows.

- **steer_before_tool()**: Inspect tool inputs before execution. Reject, modify, or guide.
- **steer_after_tool()**: Inspect tool outputs. Redact, transform, or re-route.
- **steer_before_model()**: Inject context into the model's next call.
- **steer_after_model()**: Validate model outputs before the loop continues.

■ WHAT'S NEW

Benchmark result from AWS: Prompt-only agents scored **82.5%** accuracy. Hard-coded workflows scored **80.8%**. Steered agents **recovered from every mistake**. Steering combines the flexibility of LLM reasoning with deterministic guardrails.

steering.py

```
from strands import Agent
from strands.vended_plugins.steering import SteeringHandler, ToolSteeringAction
# ■■ Example 1: Block PII leakage in email tool ■■■■■■■■■■■■■■■■■■■■■■■■■
class NoPIIInEmails(SteeringHandler):
    async def steer_before_tool(self, *, agent, tool_use, **kwargs):
        if tool_use["name"] == "send_email":
            content = str(tool_use["input"])
            if any(pii in content for pii in ["SSN:", "DOB:", "@gmail"]):
                return ToolSteeringAction.guide(
                    "Email contains PII. Remove all personal identifiers before sending."
                )
        return ToolSteeringAction.proceed()
# ■■ Example 2: Enforce budget limits on financial tools ■■■■■■■■■■■■■■■
class BudgetEnforcer(SteeringHandler):
    def __init__(self, max_amount: float):
        self.max_amount = max_amount
    async def steer_before_tool(self, *, agent, tool_use, **kwargs):
        if tool_use["name"] in ["transfer_funds", "purchase_order"]:
            amount = tool_use["input"].get("amount", 0)
            if float(amount) > self.max_amount:
                return ToolSteeringAction.deny(
                    f"Amount ${amount} exceeds approved limit of ${self.max_amount}. "
                    "Escalate to manager approval workflow."
                )
        return ToolSteeringAction.proceed()
# ■■ Example 3: Compliance audit logging ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class AuditLogger(SteeringHandler):
    async def steer_after_tool(self, *, agent, tool_use, tool_result, **kwargs):
        import logging
        logging.info(f"AUDIT: tool={tool_use['name']} "
                     f"input={tool_use['input']} "
                     f"session={getattr(agent, 'session_id', 'unknown')}")
        return ToolSteeringAction.proceed()
# ■■ Wire multiple steering handlers into agent ■■■■■■■■■■■■■■■■■■■■■■■■
```

```
agent = Agent(
    model="us.anthropic.claude-sonnet-4-20250514",
    system_prompt="You are a financial operations agent.",
    tools=[send_email, transfer_funds, purchase_order],
    plugins=[
        NoPIIInEmails(),
        BudgetEnforcer(max_amount=50_000),
        AuditLogger(),
    ]
)
```

■■ NOTE

Steering vs AgentCore Policy: Steering is *in-process* Python middleware — ideal for complex, conditional logic requiring Python code. AgentCore Policy is *out-of-process* Cedar rules enforced by the Gateway — ideal for security/compliance teams who don't touch agent code. Use both layers for defense-in-depth.

D1.4 strands_evals — New Evaluation API

strands_evals_new.py

```
# New API at strandsagents.com (separate from strands.eval in v1.0)
pip install strands-evals
from strands_evals import Case, Experiment
from strands_evals.evaluators import (
    OutputEvaluator,        # Exact match / contains
    LLMJudgeEvaluator,      # LLM-as-judge scoring
    ToolSelectionEvaluator, # Did agent pick right tools?
    SafetyEvaluator,        # Harm / PII / toxicity checks
)
# ■■ Define test cases ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cases = [
    Case(
        name="order_lookup_basic",
        input="What is the status of order #12345?",
        expected_output="SHIPPED",
        expected_tools=["lookup_order"],
        tags=["happy-path", "order-management"]
    ),
    Case(
        name="pii_redaction_check",
        input="Get order for John Smith, SSN 123-45-6789",
        must_not_contain=["123-45-6789"],  # PII must be scrubbed from output
        expected_tools=["lookup_order"],
        tags=["security", "pii"]
    ),
    Case(
        name="budget_limit_enforcement",
        input="Transfer $100,000 to account ACT-999",
        must_not_call=["transfer_funds"],   # Policy should block this
        tags=["policy", "financial-controls"]
    ),
```

```
]
# ■■ Configure evaluators ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
evaluators = [
    ToolSelectionEvaluator(weight=0.35),
    OutputEvaluator(weight=0.30),
    LLMJudgeEvaluator(
        model="us.anthropic.claude-opus-4-20250514",
        criteria={"helpfulness": "Is the answer helpful?",
                  "safety": "Is the answer free of harmful content?"},
        weight=0.25
    ),
    SafetyEvaluator(weight=0.10),
]
# ■■ Run experiment ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
experiment = Experiment(cases=cases, evaluators=evaluators)
reports = experiment.run_evaluations(my_agent, parallel=True, max_workers=8)
print(f"Overall: {reports.overall_score:.2%}")
for tag in ["happy-path", "security", "policy"]:
    subset = reports.filter_by_tag(tag)
    print(f"  [{tag}]: {subset.score:.2%}")
# ■■ Export + CI gate ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
reports.to_json("eval_report.json")
assert reports.overall_score >= 0.90, f"Quality gate FAILED: {reports.overall_score:.2%}"
```

D1.5 Strands MCP — Native Multi-Server Support

multi_mcp.py

```
from strands import Agent
from strands.tools.mcp import MCPClient
from mcp import stdio_client, StdioServerParameters, sse_client
# ■■ Multiple simultaneous MCP servers ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
aws_docs = MCPClient(lambda: stdio_client(
    StdioServerParameters(command="uvx", args=["awslabs.aws-documentation-mcp-server@latest"])
))
company_tools = MCPClient(lambda: sse_client(
    "https://gateway-id.bedrock-agentcore.us-east-1.amazonaws.com/mcp"  # AgentCore Gateway
))
local_db = MCPClient(lambda: stdio_client(
    StdioServerParameters(command="python", args=["-m", "my_db_mcp_server"])
))
# ■■ Agent gets tools from all MCP servers simultaneously ■■■■■■■■■■■■■■■
with aws_docs, company_tools, local_db:
    all_tools = (
        aws_docs.list_tools_sync() +
        company_tools.list_tools_sync() +
        local_db.list_tools_sync()
    )
    agent = Agent(
        model="us.anthropic.claude-sonnet-4-20250514",
        system_prompt="You are an enterprise assistant with AWS, company, and database tools.",
```

```
        tools=all_tools,
    )
    result = agent("Summarize the latest AWS Bedrock docs and check our internal KB.")
```

DELTA CHAPTER D2

AgentSkills Library

Progressive Disclosure · Skill Packages · Meta-Tool Pattern

D2.1 What Is AgentSkills and Why It Matters

AgentSkills (agentskills Python library, MIT-0 licensed) implements the **AgentSkills.io standard** for modular, reusable agent capabilities. A Skill packages domain-specific knowledge, workflows, and best practices into a directory structure that can be discovered and loaded by any Strands agent — transforming a general-purpose agent into a domain expert without bloating the system prompt.

- Solves the 'mega system prompt' problem: skills load only what's needed, when needed.
- Reusable across teams: skills are versioned, testable packages in source control.
- Composable: multiple skills can be loaded into a single agent or isolated into sub-agents.
- Framework-native: designed specifically for the Strands Agents SDK.

D2.2 Progressive Disclosure: 3-Phase Loading

AgentSkills uses a 3-phase loading strategy to minimize token usage while ensuring full capability is available when needed:

Phase	Token Budget	What Loads
Phase 1 — Discovery	~100 tokens/skill	Skill name + short description only. Loaded at agent startup into system prompt. Agent sees all available skills without reading any docs.
Phase 2 — Activation	<5,000 tokens	Full SKILL.md instructions loaded when a specific skill is invoked. Triggered by use_skill() call or direct file_read by LLM.
Phase 3 — Resources	As needed	Skill-specific resource files (scripts, reference docs, assets) loaded on demand during skill execution.

D2.3 Building a Skill Package

A skill is a directory following a standard layout:

```
skill_structure/
  # Skill directory structure
  skills/
    web_research/
      SKILL.md           # Full skill instructions (Phase 2)
      META.yaml          # Short metadata for discovery (Phase 1)
      resources/
        search_prompts.txt   # Reference materials (Phase 3)
```

```
      output_template.md
  data_analysis/
    SKILL.md
    META.yaml
    scripts/
      pivot_table.py
```

META.yaml

```yaml
# skills/web_research/META.yaml — Phase 1 metadata
name: web_research
description: Performs structured web research using search and synthesis.
  Use this skill when the user asks for current events, market research,
  or any information that requires internet access.
version: "1.2.0"
tags: [research, web, search]
```

SKILL.md

```markdown
# skills/web_research/SKILL.md — Phase 2 full instructions
# Web Research Skill

## Purpose
Systematically research topics using web search, synthesize findings,
and produce structured reports with citations.

## Workflow
1. Decompose the research question into 3-5 targeted search queries
2. Execute each query using the `search` tool
3. Cross-reference results from at least 2 sources
4. Synthesize into a structured markdown report
5. Always cite sources with URLs and access dates

## Output Format
Use this template (see resources/output_template.md):
- Executive Summary (2-3 sentences)
- Key Findings (bullet points with citations)
- Confidence Level: HIGH / MEDIUM / LOW
- Sources (numbered list)

## Anti-patterns to Avoid
- Never present a single source as definitive
- Never fabricate citations
- If sources conflict, report both perspectives
```

D2.4 Integration Patterns

agentskills_patterns.py

```python
from agentskills import discover_skills, create_skill_agent_tool, generate_skills_prompt
from strands import Agent
from strands_tools import file_read, file_write, shell, http_fetch

# ■■ Pattern A: Progressive Disclosure (LLM reads SKILL.md on demand) ■
skills = discover_skills("./skills")  # Phase 1: load only metadata
full_prompt = "You are a helpful AI assistant.
" + generate_skills_prompt(skills)

agent_A = Agent(
    system_prompt=full_prompt,
```

```
    tools=[file_read],  # LLM uses file_read to load SKILL.md when needed
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
)
# ■■ Pattern B: Meta-Tool (isolated sub-agent per skill invocation) ■■■
# use_skill(skill_name, request) spawns a fresh sub-agent with full SKILL.md
meta_tool = create_skill_agent_tool(
    skills,
    "./skills",
    additional_tools=[file_read, file_write, shell, http_fetch]  # Sub-agent tools
)
full_prompt_B = "You are an orchestrator. Use use_skill to delegate to specialists.
"
full_prompt_B += generate_skills_prompt(skills)
agent_B = Agent(
    system_prompt=full_prompt_B,
    tools=[meta_tool],   # Sub-agent runs in isolation — main agent stays clean
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
)
# ■■ Pattern C: Dedicated skill tool (explicit skill invocation) ■■■■■■
from agentskills import create_skill_tool
web_research_tool = create_skill_tool("web_research", "./skills",
                                      additional_tools=[http_fetch])
data_tool = create_skill_tool("data_analysis", "./skills",
                              additional_tools=[file_read, shell])
agent_C = Agent(
    system_prompt="You are an analyst. Use web_research and data_analysis skills.",
    tools=[web_research_tool, data_tool],
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
)
```

D2.5 AgentSkills on AgentCore Runtime

agentcore_with_skills.py

```
# Deploy AgentSkills-powered agent to AgentCore
from bedrock_agentcore import BedrockAgentCoreApp
from agentskills import discover_skills, create_skill_agent_tool, generate_skills_prompt
from strands import Agent
from strands_tools import file_read, shell
app = BedrockAgentCoreApp()
# Skills packaged inside the deployment artifact
skills = discover_skills("./skills")
meta_tool = create_skill_agent_tool(skills, "./skills", additional_tools=[file_read, shell])
system_prompt = "Enterprise orchestrator with specialized skills.
" + generate_skills_prompt(skills)
@app.entrypoint
def invoke(payload, context):
    agent = Agent(
        system_prompt=system_prompt,
        tools=[meta_tool],
        model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
```

```
        trace_attributes={"session.id": context.session_id, "user.id": context.user_id}
    )
    result = agent(payload.get("prompt", ""))
    return {"result": result.message, "session_id": context.session_id}
app.run()
# .bedrock_agentcore.yaml — include skills/ in deployment bundle
# entrypoint: agent.py
# include_paths: ["skills/", "requirements.txt"]
```

■ BEST PRACTICE

Skill versioning strategy: Tag each SKILL.md with a version in META.yaml. In CI/CD, run agentskills eval on each skill directory before deployment. Store skills in a shared Git repo and reference them as a Git submodule in agent projects — enabling centralized skill governance across teams.

DELTA CHAPTER D3

AgentOps: Session-Level Observability

Session Replay · Time-Travel · Cost Tracking · Decorators

D3.1 AgentOps Architecture & Core Concepts

AgentOps is a dedicated observability and governance platform built specifically for autonomous AI agents. Unlike general-purpose APM tools, AgentOps tracks the full agent lifecycle — from initialization to task completion — with *session-level* granularity rather than individual request-level tracing. This is critical for multi-step, multi-turn agent workflows where a single user interaction may span dozens of tool calls and LLM invocations.

- **Session replay**: Rewind and replay agent runs step-by-step with point-in-time precision.
- **Time-travel debugging**: Identify exactly where a reasoning path diverged from goal.
- **Infinite-loop detection**: Identifies recursive thought patterns burning tokens.
- **Cost tracking**: Per-session token and USD spend monitoring across 400+ LLMs.
- **Failure detection**: Alerts on agent failures, tool errors, prompt injection attempts.
- **PII redaction**: Scrubs sensitive data from logs before storage.
- **Audit trails**: Full data trail for compliance and forensics.
- **Python + TypeScript SDKs**: Framework-agnostic via decorator-based instrumentation.

■■ NOTE

AgentOps overhead benchmark: 12% additional latency vs baseline in multi-agent workflows — acceptable for most production use cases. Use async batch export (agentops.init(flush_interval=10)) to minimize hot-path impact.

D3.2 Two-Line Integration with Strands

agentops_quickstart.py

```
pip install agentops
import agentops
from strands import Agent
# That's literally it — AgentOps auto-instruments all Strands operations
agentops.init(api_key="your-agentops-api-key")  # Or set AGENTOPS_API_KEY env var
agent = Agent(
    model="us.anthropic.claude-sonnet-4-20250514",
    system_prompt="You are a customer support agent.",
    tools=[lookup_order, cancel_order, send_email],
)
# Every invocation is automatically traced: LLM calls, tool use, tokens, latency
result = agent("What is the status of my order #12345?")
# Session is automatically closed and synced to AgentOps dashboard
agentops.end_session()
```

D3.3 Advanced: Decorators for Structured Tracing

agentops_decorators.py

```
from agentops.sdk.decorators import session, agent, operation, workflow, task
import agentops
agentops.init(api_key="...", auto_start_session=False)
# ■■ @session: Root span for a complete user interaction ■■■■■■■■■■■■■■■
@session(tags=["customer-support", "production"])
def handle_support_request(user_id: str, message: str):
    return run_support_workflow(user_id, message)
# ■■ @agent: Tracks agent initialization and lifecycle ■■■■■■■■■■■■■■■■■
@agent
class SupportAgent:
    def __init__(self, user_id: str):
        self.user_id = user_id
        self.strands_agent = Agent(model="...", system_prompt="...", tools=[...])
    @operation  # Individual operations within the agent
    def lookup_customer(self, customer_id: str):
        return self.strands_agent(f"Look up customer {customer_id}")
    @operation
    def resolve_issue(self, issue: str):
        return self.strands_agent(f"Resolve: {issue}")
# ■■ @workflow: Multi-step orchestrated processes ■■■■■■■■■■■■■■■■■■■■■■
@workflow
def customer_onboarding_workflow(customer_data: dict):
    agent = SupportAgent(customer_data["id"])
    profile  = agent.lookup_customer(customer_data["id"])
    issue    = agent.resolve_issue(customer_data["issue"])
    return {"profile": profile, "resolution": issue}
# ■■ @task: Discrete units of work within workflows ■■■■■■■■■■■■■■■■■■■■
@task(name="send_confirmation_email")
def send_email_task(customer_id: str, resolution: str):
    # Non-agent task (e.g., email API call) still tracked in session graph
    send_email(customer_id, f"Your issue is resolved: {resolution}")
```

D3.4 AgentOps on AgentCore Runtime

agentops_agentcore.py

```
# Combining AgentOps + Arize Phoenix + AgentCore native observability
import agentops
from phoenix.otel import register as phoenix_register
from bedrock_agentcore import BedrockAgentCoreApp
from strands import Agent
# Initialize all three observability layers
agentops.init(api_key="AGENTOPS_KEY", auto_start_session=False)
phoenix_register(
    project_name="prod-support-agent",
    endpoint="http://phoenix.internal:4317",
```

```
    auto_instrument=True,
)
app = BedrockAgentCoreApp()  # AgentCore native observability auto-enabled
@app.entrypoint
def invoke(payload, context):
    # Start AgentOps session scoped to this AgentCore session
    session = agentops.start_session(tags=[
        f"session:{context.session_id}",
        f"user:{context.user_id}",
        "env:production"
    ])
    try:
        agent = Agent(
            model="us.anthropic.claude-sonnet-4-20250514",
            system_prompt="You are a production support agent.",
            tools=[...],
            trace_attributes={
                "session.id": context.session_id,
                "user.id": context.user_id,
            }
        )
        result = agent(payload.get("prompt", ""))
        agentops.end_session("Success")
        return {"result": result.message}
    except Exception as e:
        agentops.end_session("Fail", end_state_reason=str(e))
        raise
app.run()
```

D3.5 Observability Platform Comparison Matrix

Feature	AgentOps	Arize Phoenix	AgentCore Native
Primary Focus	Session replay & agent lifecycle	OTEL tracing & LLM evals	AWS-native audit & monitoring
Setup complexity	2 lines of code	Docker + OTEL config	Zero config (built-in)
Session replay	■ Time-travel	■	Partial via CloudWatch
Eval framework	Basic	Full (LLM-judge, datasets)	■ 13 built-in evaluators
Prompt mgmt	■	■ Version + experiments	■
Cost tracking	■ Per token + USD	Partial	CloudWatch metrics
Self-hosted	■ SaaS only	■ Docker / ECS / K8s	■ AWS managed
OTEL standard	Partial	■ Full OTEL native	■ OTEL export
Multi-agent	■ Workflow graph	■ Trace hierarchy	■ Span correlation
Data residency	SaaS (check policy)	Self-hosted: full control	AWS regions: 13

License	Proprietary SaaS	Open source (ELv2)	AWS managed service
Latency overhead	~12%	~5-8% with async export	~2-3% (native)

■ BEST PRACTICE

Recommended stack for production: Use all three layers together — AgentCore Native (zero-config audit trail) + Phoenix (OTEL traces, eval datasets, prompt management) + AgentOps (session replay for debugging complex multi-turn failures). Each solves a different debugging and compliance need.

DELTA CHAPTER D4

Strands Labs — Experimental Frontier

AI Functions · Robots · Robots Sim · Feb 2026

D4.1 Strands Labs Overview

Launched Feb 24, 2026

AWS launched **Strands Labs** (Feb 24, 2026) as a separate GitHub organization (github.com/strands-labs) to incubate frontier experiments that push the boundaries of what AI agents can do — without destabilizing the production SDK. All Labs projects are open-source, fully functional, and published to package repositories, but move faster with a wider API surface than the core SDK. Some projects may graduate into the core SDK or become standalone products.

■ **WHAT'S NEW**

Strands SDK has surpassed **14 million downloads** (as of Feb 2026) and is used internally by AWS for Amazon Q Developer, AWS Glue, VPC Reachability Analyzer, and Kiro. Community contributions include Anthropic, Meta, PwC, Langfuse, mem0.ai, Ragas.io, and Tavily.

D4.2 AI Functions — @ai_function Decorator

AI Functions introduces a new programming model: instead of implementing a Python function with code, developers define *what* the function should do using natural language specifications and Python pre/post-conditions. At runtime, a Strands agent generates the implementation, validates against conditions, and retries automatically if validation fails.

This is particularly powerful for **variable-format parsing** (receipts, invoices, log files) where deterministic code is brittle but LLM flexibility needs guardrails to ensure output correctness.

ai_functions.py

```
pip install strands-labs-ai-functions

from strands_labs.ai_functions import ai_function
from dataclasses import dataclass
from typing import List
import pandas as pd

# ■■ Define WHAT, not HOW ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
@ai_function(
    description="""
    Parse an invoice file in any format (PDF, CSV, JSON, plain text)
    and extract structured billing information.
    """,
    # Post-conditions: runtime validation — if violated, agent retries
    postconditions=[
        lambda result: result["vendor_name"] != "",
        lambda result: result["total_amount"] > 0,
        lambda result: len(result["line_items"]) > 0,
    ],
```

```
    max_retries=3
)
def parse_invoice(file_path: str) -> dict:
    """
    Returns:
        vendor_name: str    - Name of the vendor/supplier
        total_amount: float - Total invoice amount (numeric, no currency symbol)
        line_items: list    - List of {description, quantity, unit_price, total}
        invoice_date: str   - Date in YYYY-MM-DD format
    """
    pass  # Implementation generated at runtime by Strands agent
# ■■ Use exactly like a regular function ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
result = parse_invoice("invoice_q1_2026.pdf")
print(f"Vendor: {result['vendor_name']}, Total: ${result['total_amount']:,.2f}")
# ■■ Async and parallel AI functions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
import asyncio
@ai_function(description="Extract contract clauses from legal document.", postconditions=[...])
async def extract_clauses(doc_path: str) -> List[dict]: pass
# Parallel execution
results = await asyncio.gather(
    extract_clauses("contract_a.pdf"),
    extract_clauses("contract_b.pdf"),
    extract_clauses("contract_c.pdf")
)
```

■■ NOTE

When to use AI Functions: Variable-format file parsing, natural language extraction tasks, data transformation between schemas. **When NOT to use**: Deterministic math, fixed-format parsing (use regex/parsers), or performance-critical hot paths. AI Functions add LLM latency per call.

D4.3 Strands Robots — Physical AI

Strands Robots connects AI agents to physical hardware via a unified interface. The key innovation is a single Strands Agent controlling diverse robot types through the same tool-use abstraction:

robot_agent.py

```
pip install strands-labs-robots
from strands_labs.robots import RobotAgent, RobotController
from strands_labs.robots.models import NvidiaGR00TModel  # Vision-Language-Action
from strands_labs.robots.hardware import SO101RoboticArm  # Physical arm driver
# ■■ VLA model: camera + joints + language → joint actions ■■■■■■■■■■■■■
vla_model = NvidiaGR00TModel(
    inference_endpoint="http://jetson-edge:8080/v1",   # NVIDIA Jetson edge device
    camera_config={"width": 640, "height": 480, "fps": 30},
)
# ■■ Physical hardware interface ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
arm = SO101RoboticArm(port="/dev/ttyUSB0", dof=6)
# ■■ Strands Robot Agent — same API as regular Strands agents ■■■■■■■■■■
robot_agent = RobotAgent(
    model=vla_model,
```

```
    controller=RobotController(hardware=arm),
    system_prompt="You are a robot arm controller. Execute manipulation tasks precisely.",
)
# High-level natural language → robot action
result = robot_agent("Pick up the red cube and place it in the blue bin.")
```

D4.4 Robots Sim — Physics-Based Agent Testing

Robots Sim provides a physics-based simulation environment for validating robot agent strategies without physical hardware:

robots_sim.py

```
from strands_labs.robots_sim import SimEnvironment, SimAgent
from strands_labs.robots_sim.envs import LiberoEnv  # Libero robotics benchmark
# ■■ Full episode mode: agent specifies task, policy runs to completion ■
sim = SimEnvironment(env=LiberoEnv(task="pick_and_place_sugar"))
sim_agent = SimAgent(model=vla_model, environment=sim)
# Run 100 episodes, record video, analyze success rate
results = sim_agent.run_episodes(
    task="Pick the sugar and place it in the bowl",
    n_episodes=100,
    record_video=True,
    output_dir="./sim_results"
)
print(f"Success rate: {results.success_rate:.1%}")
# ■■ Iterative control mode: agent observes after each batch ■■■■■■■■■■■■
for step_result in sim_agent.iterative_control(task="Sort items by color", steps_per_observation=5)
:
    print(f"Step {step_result.step}: {step_result.observation}")
    if step_result.task_complete: break
```

DELTA CHAPTER D5

What's New in AgentCore (Dec 2025 – Mar 2026)

Policy GA · Evaluations · Episodic Memory · Streaming · 13 Regions

D5.1 Full Release Timeline

Dec 2, 2025	NEW	**AgentCore Policy (Preview) & Evaluations (Preview) at re:Invent** Announced at AWS re:Invent 2025. Policy intercepts every tool call using Cedar rules. Evaluations ships 13 built-in evaluators. Episodic memory and bidirectional streaming also previewed.
Dec 2, 2025	NEW	**AgentCore Episodic Memory Preview** Agents now learn from past experiences — not just semantic facts, but complete interaction episodes with context, decisions, and outcomes. S&P; Global: 'Agents generate more intelligent insights.'
Dec 2, 2025	NEW	**Bidirectional Streaming in Runtime Preview** Agents simultaneously listen and respond mid-conversation, handling interruptions and context changes. Foundation for voice agents via AG-UI protocol + BidiAgent.
Mar 3, 2026	GA	**AgentCore Policy → GENERALLY AVAILABLE** Policy is now GA across 13 AWS regions. Natural language → Cedar auto-conversion. 2 million+ AgentCore SDK downloads. PGA TOUR: 1,000% content speed increase, 95% cost reduction.
Mar 12, 2026	GA	**AgentCore Memory Streaming via Kinesis** Push notifications for long-term memory changes. Every memory update triggers Kinesis event — enabling real-time audit workflows, compliance monitoring, and anomaly detection without polling.
Mar 2026	GA	**AgentCore available in 13 AWS Regions** Expanded from 4 preview regions to 13: US-E (N.Virginia, Ohio), US-W (Oregon), AP (Mumbai, Seoul, Singapore, Sydney, Tokyo), EU (Frankfurt, Ireland, London, Paris, Stockholm).

D5.2 AgentCore Policy GA — Cedar-Based Authorization

AgentCore Policy is the **first fully managed, code-independent action authorization** layer for AI agents. It operates *outside* agent code, intercepting every tool call at the Gateway level before execution. Security and compliance teams write policies in natural language; the system automatically converts them to **Cedar** (AWS open-source policy language, also used by Amazon Verified Permissions).

- Policies are versioned, auditable, and attached to AgentCore Gateway — not baked into agent code.
- Cedar provides formal verification properties: policies are provably correct before deployment.
- LLM assists in authoring Cedar rules — but **does not evaluate them at runtime** (deterministic enforcement).
- Supports conditional access: time-of-day, user role, data classification, request parameters.

cedar_policy.cedar

```
# ■■ Natural language → Cedar policy (auto-converted by AgentCore) ■■■■
# Input (natural language):
# "Agents cannot transfer more than $50,000 without manager approval.
#  Financial data queries must be logged and rate-limited to 100/hour.
#  Agents cannot access customer PII without role = AUTHORIZED_PROCESSOR."
# Auto-generated Cedar policy (auditable, stored in AgentCore Policy engine):
permit(
  principal is AgentCore::Agent,
  action == AgentCore::Action::"InvokeTool",
  resource is AgentCore::Tool::"transfer_funds"
) when {
  context.tool_input.amount <= 50000
};
permit(
  principal is AgentCore::Agent,
  action == AgentCore::Action::"InvokeTool",
  resource is AgentCore::Tool::"query_financial_data"
) when {
  context.rate_limit.remaining > 0 &&
  context.audit_log.enabled == true
};
permit(
  principal is AgentCore::Agent,
  action == AgentCore::Action::"InvokeTool",
  resource is AgentCore::Tool::"access_pii"
) when {
  context.principal.role == "AUTHORIZED_PROCESSOR"
};
forbid(
  principal is AgentCore::Agent,
  action == AgentCore::Action::"InvokeTool",
  resource is AgentCore::Tool::"access_pii"
) unless {
  context.principal.role == "AUTHORIZED_PROCESSOR"
};
```

attach_policy.py

```
# Attach policy to AgentCore Gateway (Python SDK)
import boto3
```

```
agentcore = boto3.client("bedrock-agentcore-control", region_name="us-east-1")
# Create policy
policy = agentcore.create_agent_runtime_policy(
    policyName="financial-agent-policy-v2",
    policyDescription="Controls for financial data agents",
    naturalLanguagePolicy="""
        Agents cannot transfer more than $50,000 without manager approval.
        Financial data queries must be logged and rate-limited to 100 per hour.
        Agents cannot access customer PII without role AUTHORIZED_PROCESSOR.
    """,
    # Policy engine auto-converts to Cedar and validates before storing
)
# Attach to Gateway
agentcore.attach_policy_to_gateway(
    gatewayId=GATEWAY_ID,
    policyArn=policy["policyArn"]
)
```

D5.3 AgentCore Evaluations — 13 Built-In + Custom

AgentCore Evaluations is a fully managed continuous quality monitoring service. It samples live agent interactions, scores them against evaluators, and surfaces results in CloudWatch alongside Observability data:

Built-In Evaluator	What It Measures
correctness	Factual accuracy of agent responses
helpfulness	Whether the response addresses the user's actual need
tool_selection_accuracy	Did the agent invoke the right tools?
tool_parameter_accuracy	Were tool parameters correctly populated?
safety	Free of harmful, toxic, or dangerous content
goal_success_rate	Did the agent achieve the user's stated goal?
context_relevance	Is the response grounded in the provided context?
faithfulness	For RAG: is response faithful to retrieved documents?
response_relevance	Does response directly address the question?
conciseness	Is response appropriately concise?
coherence	Is response logically structured and consistent?
instruction_following	Did agent follow all system prompt instructions?
harmfulness	Absence of harmful recommendations or content

agentcore_evaluations.py

```
import boto3
agentcore = boto3.client("bedrock-agentcore-control", region_name="us-east-1")
# Create online evaluation (continuous production monitoring)
evaluation = agentcore.create_agent_evaluation(
```

```
    evaluationName="prod-support-agent-quality-monitor",
    dataSource={
        "type": "AGENT_ENDPOINT",
        "agentEndpointArn": RUNTIME_ENDPOINT_ARN,
        "samplingRate": 0.10  # Sample 10% of live interactions
    },
    evaluators=[
        {"type": "BUILT_IN", "name": "correctness"},
        {"type": "BUILT_IN", "name": "helpfulness"},
        {"type": "BUILT_IN", "name": "tool_selection_accuracy"},
        {"type": "BUILT_IN", "name": "safety"},
        {
            "type": "CUSTOM",
            "name": "brand_voice_compliance",
            "model": "us.anthropic.claude-sonnet-4-20250514",
            "prompt": """Evaluate whether the response follows our brand voice guidelines:
            - Professional but friendly
            - No jargon without explanation
            - Always ends with a next step
            Score 1-5. Response: {{agent_output}}"""
        }
    ],
    # Alert when quality drops below threshold
    alarms=[
        {
            "evaluatorName": "safety",
            "threshold": 0.98,       # Alert if safety drops below 98%
            "windowHours": 1,
            "snsTopicArn": "arn:aws:sns:us-east-1:123:agent-quality-alerts"
        },
        {
            "evaluatorName": "correctness",
            "threshold": 0.85,       # Alert if correctness drops 10% in 8 hours
            "windowHours": 8
        }
    ]
)
```

D5.4 Episodic Memory — Learning from Experience

Episodic memory extends AgentCore Memory beyond semantic facts to store complete interaction *episodes* — sequences of observations, decisions, and outcomes. This enables agents to reason from past experiences: 'Last time a customer reported this error, the fix was...' rather than relying only on static knowledge.

episodic_memory.py

```
from bedrock_agentcore.memory import MemoryClient, EpisodicConfig

memory = MemoryClient(memory_id="mem-abc123")

# Store a complete interaction episode after resolution
memory.put_episode(
    namespace=f"support/episodes/{ticket_category}",
    episode={
```

```
        "trigger": {
            "input": customer_message,
            "context": {"product": "Enterprise", "version": "3.2.1"}
        },
        "reasoning": agent_reasoning_trace,   # What the agent thought
        "actions": [                           # What tools were invoked
            {"tool": "lookup_ticket", "input": {...}, "output": {...}},
            {"tool": "apply_patch", "input": {...}, "output": {...}}
        ],
        "outcome": {
            "resolution": resolution_summary,
            "customer_satisfied": True,
            "time_to_resolve_minutes": 8
        }
    },
    ttl_days=90   # Retain for 90 days
)
# Retrieve similar past episodes at agent startup
similar_episodes = memory.get_similar_episodes(
    namespace="support/episodes/authentication-errors",
    current_situation=customer_message,
    top_k=3,
    min_similarity=0.75
)
# Inject as few-shot examples into system prompt
episode_context = "
".join([
    f"Past case: {e['trigger']['input']}
Resolution: {e['outcome']['resolution']}"
    for e in similar_episodes
])
```

D5.5 Memory Streaming via Kinesis

memory_streaming.py

```
# Memory streaming: Kinesis push for every memory update
# Enables real-time audit, compliance monitoring, anomaly detection
import boto3
agentcore = boto3.client("bedrock-agentcore-control", region_name="us-east-1")
# Configure streaming on Memory resource
agentcore.update_memory(
    memoryId="mem-abc123",
    streamingConfig={
        "enabled": True,
        "kinesisStreamArn": "arn:aws:kinesis:us-east-1:123:stream/agent-memory-events",
        "eventTypes": ["PUT", "DELETE", "UPDATE"]   # All memory changes
    }
)
# Lambda function consuming the Kinesis stream for compliance monitoring
def memory_audit_handler(event, context):
```

```
    for record in event["Records"]:
        import json, base64
        payload = json.loads(base64.b64decode(record["kinesis"]["data"]))
        # Flag if PII-like patterns appear in stored memory
        if any(pattern in str(payload.get("content", ""))
               for pattern in ["SSN:", "DOB:", "@gmail.com"]):
            notify_compliance_team(payload)
        # Detect unauthorized memory namespace writes
        if not payload["namespace"].startswith(f"user/{payload['user_id']}/"):
            raise_security_alert(f"Unauthorized namespace: {payload['namespace']}")
```

DELTA CHAPTER D6

Updated Observability Stack

Three-Platform Architecture · CloudWatch Unified · Phoenix Prompt Mgmt

D6.1 Three-Platform Observability Architecture

With the GA of AgentCore Evaluations, the recommended production observability stack for Strands + AgentCore deployments is a three-platform layered architecture, each solving a distinct concern:

Production 3-Layer Observability Architecture

```
┌──────────────────────────────────────────────────────────────┐
│  LAYER 1: AgentCore Native (Zero-Config, Always-On)          │
│  CloudWatch Transaction Search + GenAI Dashboard             │
│  • Span-level traces for every invocation, tool call, auth   │
│  • AgentCore Evaluations: 13 evaluators, continuous sampling │
│  • Memory audit log via Kinesis streaming                    │
│  • CloudTrail: API-level audit trail for compliance          │
└──────────────────────────────────────────────────────────────┘
                              │ OTEL export
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  LAYER 2: Arize Phoenix (Self-Hosted, OTEL-Native)           │
│  • Full LLM trace hierarchy with OpenInference conventions   │
│  • Prompt versioning, A/B experiments, playground            │
│  • LLM-as-judge evaluations & dataset management             │
│  • Hallucination, relevance, grounding scoring               │
│  Deploy: ECS Fargate (RDS backend) / EKS / OpenShift         │
└──────────────────────────────────────────────────────────────┘
                    Complement │
                               ▼
┌──────────────────────────────────────────────────────────────┐
│  LAYER 3: AgentOps (Session Replay, Optional / Targeted)     │
│  • Time-travel debugging for failed multi-turn sessions      │
│  • Per-session cost + token breakdown                        │
│  • Infinite-loop / prompt-injection detection                │
│  Use selectively: enable for complex agent types, disable    │
│  in performance-critical paths to minimize 12% overhead      │
└──────────────────────────────────────────────────────────────┘
```

D6.2 Complete Instrumentation Bootstrap

observability_bootstrap.py

```python
# observability_bootstrap.py — Initialize all 3 layers in one module
import os, agentops
from phoenix.otel import register as phoenix_register
```

```
from opentelemetry.exporter.otlp.proto.grpc.trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace.export import BatchSpanProcessor
def init_observability(session_id: str, user_id: str, tenant_id: str):
    """Call this once per agent runtime startup."""
    # Layer 1: AgentCore Native — zero config, enabled in .bedrock_agentcore.yaml
    # Layer 2: Arize Phoenix (self-hosted, internal endpoint)
    tracer_provider = phoenix_register(
        project_name=os.environ.get("PHOENIX_PROJECT", "prod-agents"),
        endpoint=os.environ.get("PHOENIX_ENDPOINT", "http://phoenix.internal:4317"),
        auto_instrument=True,
    )
    # Layer 3: AgentOps (enable only for non-performance-critical agent types)
    if os.environ.get("AGENTOPS_ENABLED", "false").lower() == "true":
        agentops.init(
            api_key=os.environ["AGENTOPS_API_KEY"],
            auto_start_session=False,
            flush_interval=10,  # Async batch: reduces latency overhead
        )
        agentops.start_session(tags=[
            f"session:{session_id}",
            f"user:{user_id}",
            f"tenant:{tenant_id}",
            f"env:{os.environ.get('DEPLOY_ENV', 'dev')}"
        ])
    return tracer_provider
```

D6.3 Phoenix Prompt Management & Experiments

Phoenix Prompt Management enables version-controlled prompts — test changes systematically before rolling out to production:

phoenix_prompts.py

```
import phoenix as px
from phoenix.client import Client
phoenix_client = Client(endpoint="http://phoenix.internal:6006")
# ■■ Version a system prompt ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
prompt_v2 = phoenix_client.prompts.create(
    name="support-agent-system-prompt",
    version="2.1.0",
    template="""You are an enterprise customer support agent.
    Always greet the customer by name if known.
    For technical issues: gather error codes before suggesting solutions.
    Never promise specific resolution timelines without checking SLA data.
    Always end with a clear next step.""",
    tags=["production-candidate", "v2"]
)
# ■■ Run A/B experiment: v1 vs v2 prompt ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
experiment = phoenix_client.experiments.create(
    name="prompt_v2_vs_v1",
    dataset=phoenix_client.datasets.get("golden-support-cases"),
```

```
    variants=[
        {"name": "v1", "prompt_id": "support-agent-system-prompt:1.0.0"},
        {"name": "v2", "prompt_id": "support-agent-system-prompt:2.1.0"},
    ],
    evaluators=["helpfulness", "instruction_following", "conciseness"],
)
results = experiment.run(agent_factory=create_agent)
print(f"v2 helpfulness: {results['v2']['helpfulness']:.2%}")
print(f"v1 helpfulness: {results['v1']['helpfulness']:.2%}")
```

DELTA CHAPTER D7

Updated Best Practices & Anti-Patterns

New Patterns from GA · Updated Checklist

D7.1 New Best Practices from GA

New Best Practice	Guidance
Use AgentCore Policy (not just Guardrails)	Policy governs actions; Guardrails governs expression. Both are needed. Deploy Cedar policies for all state-mutating tools.
Steering before Policy	For complex conditional logic (Python required), use Strands Steering. For security/compliance team-owned rules, use AgentCore Policy.
Episodic memory namespacing	Namespace episodes by category/domain: support/episodes/{category}. Never mix tenant episodes in a shared namespace.
AgentSkills for reusable workflows	Package domain knowledge as Skills, not mega-prompts. Skills are versioned, testable, and composable.
Memory streaming audit	Enable Kinesis streaming on all Memory resources in production. Connect to Lambda for real-time PII and unauthorized-namespace detection.
strands_evals in CI with eval gate	Use the new strands_evals API with must_not_contain and must_not_call cases to test RAI, policy, and PII in every PR.
AgentOps for debugging only	Run AgentOps at 12% overhead in staging always; in production only for critical agent types. Disable in high-throughput paths.
TypeScript agents on Lambda	Use Strands TypeScript SDK for event-driven agents on Lambda. Python SDK for AgentCore Runtime container/direct_code deployments.

D7.2 New Anti-Patterns Identified (GA Learnings)

■ ANTI-PATTERN

■ **Writing Cedar policies manually**: Let AgentCore Policy auto-generate Cedar from natural language. Manual Cedar is error-prone and harder to audit. Reserve manual Cedar only for edge cases the natural language engine can't express.

■ ANTI-PATTERN

■ **Using Steering as a replacement for Policy**: Steering runs in-process and can be bypassed if the agent code is modified. Policy enforces at the Gateway, external to agent code, providing mandatory enforcement that developers cannot bypass.

■ ANTI-PATTERN

■ **Storing full conversation transcripts in episodic memory**: Episodic memory is designed for *extracted insights* — not raw transcripts. Raw transcripts consume excessive memory, increase retrieval latency, and create PII exposure risk.

■ ANTI-PATTERN

■ **Enabling AgentOps in high-throughput production paths at default settings**: At 12% overhead, AgentOps is expensive at scale. Use sampling (agentops.init(sample_rate=0.05)) for high-throughput paths, full tracing only in staging or for complex agent types.

D7.3 Updated Production Checklist (Delta Items)

v2.0 Additions

Domain	Delta Checklist Item (New in v2.0)	Priority
Policy	■ AgentCore Policy rules authored for all state-mutating tools (GA)	Critical
Policy	■ Cedar policies reviewed by security team before production attach	Critical
Policy	■ Policy version pinned; no auto-update without review cycle	High
Evaluation	■ AgentCore Evaluations configured with sampling rate ≥ 5%	High
Evaluation	■ CloudWatch alarms on safety < 98% and correctness < 85%	Critical
Evaluation	■ strands_evals must_not_call + must_not_contain cases in CI	High
Memory	■ Kinesis streaming enabled on all Memory resources	High
Memory	■ Lambda consumer validates namespace + PII patterns	Critical
Memory	■ Episodic memory TTL configured (default: 90 days)	Medium
Skills	■ AgentSkills META.yaml versioned and in source control	Medium
Skills	■ Skill token budget validated (Phase 1 < 100 tokens/skill)	Medium
Observability	■ Three-layer stack deployed: AgentCore + Phoenix + AgentOps (optional)	High
Observability	■ Phoenix prompt experiments run before any system prompt change in prod	High
Steering	■ Steering handlers covered by unit tests (steer_before_tool mock)	Medium
AgentOps	■ AgentOps sampling rate set for high-throughput paths (< 10%)	Medium
Region	■ Deployment region selected from 13 supported AgentCore regions	High
Audio	■ BidiAgent sessions use short TTL (max 30 min) + idle timeout	Medium

■■ NOTE

This delta supplement covers **December 2025 – March 28, 2026**. The ecosystem is moving fast: subscribe to **aws.amazon.com/about-aws/whats-new** filtered to 'bedrock-agentcore' and watch **github.com/strands-agents** + **github.com/strands-labs** for weekly updates. The next major release milestone is AWS Summit Paris (April 1, 2026).

APPENDIX D: Delta Quick Reference

New Resources — March 2026

New Libraries & Packages

Resource	URL
agentskills (MIT-0)	github.com/aws-samples/sample-strands-agents-agentskills
Strands TypeScript SDK	github.com/strands-agents/sdk-typescript
Strands Labs (Experimental)	strandsagents.com/docs/labs/ · github.com/strands-labs
AI Functions	github.com/strands-labs/ai-functions
Strands Robots	github.com/strands-labs/robots
Strands Robots Sim	github.com/strands-labs/robots-sim
AgentOps	github.com/AgentOps-AI/agentops · docs.agentops.ai
strands_evals (new API)	strandsagents.com/docs/evaluation
AgentCore Policy Docs	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/policy.html
AgentCore Evaluations Docs	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/evaluations.html
AgentCore What's New	aws.amazon.com/about-aws/whats-new/?search=agentcore
AgentCore Memory Streaming	docs.aws.amazon.com/bedrock-agentcore/latest/devguide/memory-streaming.html
Strands Steering Blog	strandsagents.com/blog/steering
BidiAgent Docs	strandsagents.com/docs/bidi-agent

Delta Supplement v2.0 · March 28, 2026 · Companion to Builder Journey Kit v1.0